IEOR Visit-Day Matching Tool

Project handoff summary | Master of Analytics Summer Project

What the tool does

Prospective graduate students visit the IEOR department over two consecutive days and want to meet as many faculty as possible, weighted by research interest. This tool collects faculty availability and student rankings, then builds an optimal meeting schedule that maximizes preference-weighted meetings while keeping the outcome fair across students. It is built for non-technical IEOR staff: a tabbed Streamlit app where staff load data, click Match, and review and download per-student and per-faculty schedules.

Mathematical Formulation

$$\textit{Let}:$$

$$-\ S\ =\ \textit{set of students}$$

$$-\ F\ =\ \textit{set of faculty}$$

$$-\ T\ =\ \textit{set of time slots}$$

$$-\ \text{SCALE}\ =\ 10000\ \text{(integer scaling factor for percentage precision)}$$

$$\text{scaled\_floor\_weight}\ =\ \max\left(1,\ \text{round}\left(\frac{\text{floor\_weight}\ \cdot\ \text{mean}(V_{\max,s})}{\text{SCALE}}\right)\right)$$

$$\textit{Availability}:\ \ A_{f,t}\ =\ \{1\quad \text{if faculty } f \text{ is available at slot } t,\ 0\quad \text{otherwise}$$

$$\textit{Preference value (convex rank}-\textit{decay)}:$$

$$v_{s,f}\ =\ \text{round!}\left(\text{rank\_base}\ \cdot\ \text{rank\_decay}^{(\text{rank}s,f-1)}\right)$$

$$\textit{Maximum possible satisfaction per student}:$$

$$V\max,s\ =\ \sum_{i=1}^{\min(|F_s|,|T|)} v_{s,f_{(i)}}\ \text{where } F_s \text{ is the set of faculty ranked by student } s$$

$$\textit{Decision Variables}$$

$$\textit{Assignment variable}:\ \ x_{s,f,t}\ \in\ 0,1\ \text{ for all } (s,f,t) \text{ with } A_{f,t}=1$$

$$\textit{Student satisfaction}:\ \ \text{sat}s\ =\ \sum f\ \in\ F\ \sum_{t\in T} v_{s,f}\ ,\ x_{s,f,t}$$

$$\textit{Normalized fairness floor percentage}:\ \ \text{floor\_pct}\ \in\ [0,\ \text{SCALE}]$$

$$\textit{Constraints}$$

$$\textit{Faculty}:\ \textit{at most one meeting per slot}\ \sum_{s\in S} x_{s,f,t}\ \le\ 1\ \forall f\ \in\ F,\ \ t\ \in\ T$$

$$\textit{Student}:\ \textit{at most one meeting per slot}\ \sum_{f\in F} x_{s,f,t}\ \le\ 1\ \forall s\ \in\ S,\ \ t\ \in\ T$$

$$\textit{Student}-\textit{faculty pair meets at most once}\ \sum_{t\in T} x_{s,f,t}\ \le\ 1\ \forall s\ \in\ S,\ \ f\ \in\ F$$

$$\textit{Normalized fairness floor constraint (Linearized)}:\ \ \text{floor\_pct}\ \cdot\ V_{\max,s}\ \le\ \text{sat}s\ \cdot\ \text{SCALE}\ \forall s\ \in\ S \text{ where } V\max,s\ >\ 0$$

$$\textit{Objective Function}$$

$$\textit{Maximize total preference value plus weighted normalized fairness floor}:\ \ \max\ \left(\sum_{s\in S}\ \sum_{f\in F}\ \sum_{t\in T} v_{s,f}\ ,\ x_{s,f,t}+\ \text{scaled\_floor\_weight}\ \cdot\ \text{floor\_pct}\right)$$

Model Description

Here's a summary of the model that incorporates mini-examples where necessary.

The scoring rule round(rank_base * rank_decay^(rank-1)) converts each student's ranked faculty preferences into numerical values that the solver can optimize. It starts with a high base value for a rank-1 choice and applies exponential decay so lower-ranked options drop off sharply. For example, with rank_base = 100 and rank_decay = 0.6, a rank-1 preference is worth 100, rank-2 drops to 60, rank-3 to 36, and rank-4 to 22. This convex decay ensures that one top-ranked meeting is more valuable than several low-ranked ones, aligning the optimization with how students may actually prioritize faculty. The number of preferences a student submits (prefs_per_student) provides a simple upper bound on how much satisfaction they could theoretically accumulate, which is used to define the domain of the fairness floor variable.

The fairness floor is defined as an integer scale from 0 to 10,000, representing the minimum satisfaction percentage achieved by any student relative to their maximum potential satisfaction. For example, if a student with a Vmax of 200 achieves a satisfaction of 100, their individual satisfaction percentage is 50% (represented as 5000 in the integer space). The fairness floor variable is constrained to be less than or equal to every active student's individual percentage through a linearized integer constraint, forcing it to track the absolute worst-off student. Including this normalized floor in the objective encourages the solver to raise the lowest relative satisfaction percentage while still maximizing total preference value. To maintain the intended optimization balance in this 10,000x integer space, the original floor_weight parameter is automatically scaled down using the mean Vmax across all students, controlling how strongly fairness influences the final schedule without allowing a single student's constraints to stall the entire system.

The rest of the model is essentially a **bipartite matching formulation**, where students and faculty form the two sides of a bipartite graph, and meetings correspond to edges. The binary variable xs,f,t indicates whether student s meets faculty f at time t, and the constraints enforce the structure of a valid matching: each faculty can meet at most one student per slot, each student can meet at most one faculty per slot, and each student–faculty pair can meet at most once. Time slots act as an additional dimension layered onto the bipartite graph, turning the problem into a **time-indexed bipartite matching** with availability constraints. The objective then selects a set of edges that maximizes total preference value while lifting the minimum satisfaction across all students. This combination of bipartite matching, convex preference scoring, and fairness optimization produces a schedule that is structurally sound, preference-aware, and equitable.

Reasonable next steps

Roughly in priority order for the five-person team, toward the August 7 deadline:

- Wire real intake. Stand up the Google Forms for faculty availability and student rankings against the schemas in data/, complete the one-time OAuth setup (SETUP_GOOGLE.md), and move the app off synthetic data onto real collected responses.
- Deploy a shared instance. Follow DEPLOY.md to put the demo on Streamlit Community Cloud so the team and staff can click a URL instead of running locally.
- Per-meeting .ics export for each student and faculty (the ics library is already a dependency).
- Manual override in the app: let staff swap or pin a meeting after solving, then re-solve around the locked assignment.
- Stress-test fairness: build a deliberately over-subscribed instance and report how the max-min floor allocates the shortfall, so staff understand the tradeoffs the model makes.
- Harden tests: add a real pytest suite around the adapter edge cases and the solver feasibility checks so the team can gate changes in CI.